Министерство образования Республики Беларусь
УО “Брестский государственный технический университет”
Кафедра ИИТ

Лабораторная работа №3
По дисциплине: «СПП»
Вариант 9

Выполнила:
студент 3 курса
группы ПО-12
Кравченко М.А.
Проверил:
Кулик А.Д.

Брест 2026

Цель работы: приобрести навыки применения паттернов проектирования при решении практических задач с использованием языка Python.

Задание 1 (порождающий паттерн).

Проект «Бургер-закусочная». Реализовать возможность формирования заказа из определенных позиций (тип бургера (веганский, куриный и т.д.)), напиток (холодный – пепси, кока-кола и т.д.; горячий – кофе, чай и т.д.), тип упаковки – с собой, на месте. Должна формироваться итоговая стоимость заказа.

Код программы:

```
"""
Система Бургер-закусочная.
Паттерн: Builder (Строитель).
"""


class Burger:
    """Класс бургера."""

    def __init__(self, burger_type: str):
        self.burger_type = burger_type
        self.price = self._get_price()

    def _get_price(self) -> float:
        """Получить цену бургера."""
        prices = {
            "веганский": 10.6,
            "куриный": 4.3,
            "говяжий": 4.3,
            "рыбный": 9.5,
            "двойной": 8.7,
        }
        return prices.get(self.burger_type.lower(), 150)

    def get_price(self) -> float:
        """Получить цену."""
        return self.price

    def get_type(self) -> str:
        """Получить тип."""
        return self.burger_type

    def __str__(self):
        return f"{self.burger_type} бургер ({self.price} руб.)"


class Drink:
    """Класс напитка."""

    def __init__(self, drink_type: str):
        self.drink_type = drink_type
        self.price = self._get_price()

    def _get_price(self) -> float:
        """Получить цену напитка."""
        prices = {
            "фанта": 5.2,
            "кока-кола": 5.2,
            "спрайт": 5.2,
            "кофе": 6.0,
```

```
            "чай": 3.8,
            "капучино": 6.2,
        }
        return prices.get(self.drink_type.lower(), 80)

    def get_price(self) -> float:
        """Получить цену."""
        return self.price

    def get_type(self) -> str:
        """Получить тип."""
        return self.drink_type

    def __str__(self):
        return f"{self.drink_type} ({self.price} руб.)"


class Packaging:
    """Класс упаковки."""

    def __init__(self, packaging_type: str):
        self.packaging_type = packaging_type
        self.price = self._get_price()

    def _get_price(self) -> float:
        """Получить цену упаковки."""
        prices = {
            "с собой": 0,
            "на месте": 0,
            "доставка": 3.0,
        }
        return prices.get(self.packaging_type.lower(), 0)

    def get_price(self) -> float:
        """Получить цену."""
        return self.price

    def get_type(self) -> str:
        """Получить тип."""
        return self.packaging_type

    def __str__(self):
        price_str = f"+{self.price} руб." if self.price > 0 else "бесплатно"
        return f"Упаковка: {self.packaging_type} ({price_str})"


class Order:
    """Класс заказа."""

    def __init__(self):
        self.burger = None
        self.drink = None
        self.packaging = None
        self.total_price = 0

    def set_burger(self, burger: Burger):
        """Установить бургер."""
        self.burger = burger
        self._update_total()

    def set_drink(self, drink: Drink):
        """Установить напиток."""
        self.drink = drink
        self._update_total()

    def set_packaging(self, packaging: Packaging):
        """Установить упаковку."""
        self.packaging = packaging
```

```
        self._update_total()

    def _update_total(self):
        """Обновить итоговую стоимость."""
        total = 0
        if self.burger:
            total += self.burger.get_price()
        if self.drink:
            total += self.drink.get_price()
        if self.packaging:
            total += self.packaging.get_price()
        self.total_price = total

    def get_total(self) -> float:
        """Получить итоговую сумму."""
        return self.total_price

    def __str__(self):
        lines = ["\n" + "=" * 40, "ВАШ ЗАКАЗ:", "=" * 40]
        if self.burger:
            lines.append(f"Бургер: {self.burger}")
        if self.drink:
            lines.append(f"Напиток: {self.drink}")
        if self.packaging:
            lines.append(f"{self.packaging}")
        lines.append("-" * 40)
        lines.append(f"ИТОГО: {self.total_price} руб.")
        lines.append("=" * 40)
        return "\n".join(lines)


class OrderBuilder:
    """Строитель заказа (паттерн Builder)."""

    def __init__(self):
        self.order = Order()

    def choose_burger(self):
        """Выбрать бургер."""
        print("\nДоступные бургеры:")
        burgers = ["веганский", "куриный", "говяжий", "рыбный", "двойной"]
        for i, b in enumerate(burgers, 1):
            print(f"  {i}. {b}")

        choice = int(input("Выберите бургер (1-5): "))
        if 1 <= choice <= len(burgers):
            self.order.set_burger(Burger(burgers[choice - 1]))
            print(f"Добавлен: {burgers[choice - 1]} бургер")
        return self

    def choose_drink(self):
        """Выбрать напиток."""
        print("\nДоступные напитки:")
        drinks = ["пепси", "кока-кола", "спрайт", "кофе", "чай", "капучино"]
        for i, d in enumerate(drinks, 1):
            print(f"  {i}. {d}")

        choice = int(input("Выберите напиток (1-6): "))
        if 1 <= choice <= len(drinks):
            self.order.set_drink(Drink(drinks[choice - 1]))
            print(f"Добавлен: {drinks[choice - 1]}")
        return self

    def choose_packaging(self):
        """Выбрать упаковку."""
        print("\nТип упаковки:")
        pack_types = ["с собой", "на месте", "доставка"]
        for i, p in enumerate(pack_types, 1):
```

```
            print(f"  {i}. {p}")

        choice = int(input("Выберите упаковку (1-3): "))
        if 1 <= choice <= len(pack_types):
            self.order.set_packaging(Packaging(pack_types[choice - 1]))
            print(f"Выбрано: {pack_types[choice - 1]}")
        return self

    def get_order(self) -> Order:
        """Вернуть готовый заказ."""
        return self.order

    def build(self) -> Order:
        """Вернуть готовый заказ (алиас)."""
        return self.order


class OrderDirector:
    """Директор для создания предустановленных заказов."""

    def __init__(self):
        self.builder = OrderBuilder()

    def create_standard_burger_order(self) -> Order:
        """Стандартный заказ."""
        self.builder = OrderBuilder()
        order = self.builder.choose_burger().choose_drink().choose_packaging().get_order()
        return order

    def create_vegan_order(self) -> Order:
        """Веганский заказ."""
        order = Order()
        order.set_burger(Burger("веганский"))
        order.set_drink(Drink("чай"))
        order.set_packaging(Packaging("на месте"))
        return order


def main():
    """Основная функция."""
    print("=" * 60)
    print("БУРГЕР-ЗАКУСОЧНАЯ")
    print("=" * 60)

    print("\nСоздание нового заказа:")

    builder = OrderBuilder()
    builder.choose_burger()
    builder.choose_drink()
    builder.choose_packaging()

    order = builder.build()
    print(order)

    print("\n1. Оформить заказ")
    print("2. Отменить")
    choice = input("Выбор: ")

    if choice == "1":
        print("\nЗаказ оформлен. Спасибо!")
    else:
        print("\nЗаказ отменен.")


if __name__ == "__main__":
    main()
```

```
Создание нового заказа:

Доступные бургеры:
  1. веганский
  2. куриный
  3. говяжий
  4. рыбный
  5. двойной
Выберите бургер (1-5): 5
Добавлен: двойной бургер

Доступные напитки:
  1. пепси
  2. кока-кола
  3. спрайт
  4. кофе
  5. чай
  6. капучино
Выберите напиток (1-6): 3
Добавлен: спрайт

Тип упаковки:
  1. с собой
  2. на месте
  3. доставка
Выберите упаковку (1-3): 3
Выбрано: доставка

========================================
ВАШ ЗАКАЗ:
========================================
Бургер: двойной бургер (8.7 руб.)
Напиток: спрайт (5.2 руб.)
Упаковка: доставка (+3.0 руб.)
----------------------------------------
ИТОГО: 16.9 руб.
========================================
```

Задание 2 (структурный паттерн).

Проект «Часы». В проекте должен быть реализован класс, который дает возможность пользоваться часами со стрелками так же, как и цифровыми часами. В классе «Часы со стрелками» хранятся повороты стрелок.

Код программы:

```
"""
Проект Часы.
Структурный паттерн: Adapter (Адаптер).
"""


class AnalogClock:
    """Часы со стрелками. Хранит повороты стрелок в градусах."""

    def __init__(self, hours_angle: float = 0, minutes_angle: float = 0):
        self.hours_angle = hours_angle % 360
        self.minutes_angle = minutes_angle % 360
        self._sync_time()

    def _sync_time(self):
        """Синхронизирует время на основе углов стрелок."""
        self.hours = (self.hours_angle / 30) % 12
        self.minutes = (self.minutes_angle / 6) % 60
        self.seconds = 0

    def set_time_by_angles(self, hours_angle: float, minutes_angle: float):
        """Установить время по углам стрелок."""
        self.hours_angle = hours_angle % 360
        self.minutes_angle = minutes_angle % 360
        self._sync_time()
```

```
    def set_hours_angle(self, angle: float):
        """Установить угол часовой стрелки."""
        self.hours_angle = angle % 360
        self._sync_time()

    def set_minutes_angle(self, angle: float):
        """Установить угол минутной стрелки."""
        self.minutes_angle = angle % 360
        self._sync_time()

    def get_hours_angle(self) -> float:
        """Получить угол часовой стрелки."""
        return self.hours_angle

    def get_minutes_angle(self) -> float:
        """Получить угол минутной стрелки."""
        return self.minutes_angle

    def get_time(self):
        """Получить время в формате (часы, минуты)."""
        return int(self.hours), int(self.minutes)

    def __str__(self):
        hours_int = int(self.hours)
        minutes_int = int(self.minutes)
        return (
            f"Аналоговые часы: {hours_int:02d}:{minutes_int:02d} "
            f"(часы: {self.hours_angle:.1f}°, минуты: {self.minutes_angle:.1f}°)"
        )


class DigitalClockInterface:
    """Интерфейс цифровых часов."""

    def display_time(self) -> str:
        """Отобразить время в цифровом формате."""
        raise NotImplementedError

    def set_time(self, hours: int, minutes: int):
        """Установить время."""
        raise NotImplementedError


class DigitalClock(DigitalClockInterface):
    """Цифровые часы."""

    def __init__(self, hours: int = 0, minutes: int = 0):
        self.hours = hours % 24
        self.minutes = minutes % 60

    def display_time(self) -> str:
        """Отобразить время."""
        return f"{self.hours:02d}:{self.minutes:02d}"

    def set_time(self, hours: int, minutes: int):
        """Установить время."""
        self.hours = hours % 24
        self.minutes = minutes % 60

    def __str__(self):
        return f"Цифровые часы: {self.display_time()}"


class AnalogToDigitalAdapter(DigitalClockInterface):
    """Адаптер, позволяющий использовать аналоговые часы как цифровые."""

    def __init__(self, analog_clock: AnalogClock):
```

```
        self.analog_clock = analog_clock

    def display_time(self) -> str:
        """Отобразить время из аналоговых часов в цифровом формате."""
        hours, minutes = self.analog_clock.get_time()
        return f"{hours:02d}:{minutes:02d}"

    def set_time(self, hours: int, minutes: int):
        """Установить время на аналоговых часах через цифровой интерфейс."""
        hours_angle = (hours % 12) * 30
        minutes_angle = minutes * 6
        self.analog_clock.set_time_by_angles(hours_angle, minutes_angle)

    def get_analog_clock(self) -> AnalogClock:
        """Получить оригинальные аналоговые часы."""
        return self.analog_clock


def main():
    """Основная функция."""
    print("=" * 60)
    print("ПРОЕКТ ЧАСЫ - АДАПТЕР")
    print("=" * 60)

    print("\n1. СОЗДАНИЕ АНАЛОГОВЫХ ЧАСОВ")
    analog = AnalogClock()
    print(analog)

    print("\n2. УСТАНОВКА ВРЕМЕНИ НА АНАЛОГОВЫХ ЧАСАХ")
    hours = int(input("Введите часы (0-12): "))
    minutes = int(input("Введите минуты (0-60): "))
    analog.set_time_by_angles(hours * 30, minutes * 6)
    print(analog)

    print("\n3. СОЗДАНИЕ АДАПТЕРА")
    adapter = AnalogToDigitalAdapter(analog)
    print(f"Адаптер показывает время: {adapter.display_time()}")

    print("\n4. УСТАНОВКА ВРЕМЕНИ ЧЕРЕЗ АДАПТЕР")
    new_hours = int(input("Введите новые часы (0-23): "))
    new_minutes = int(input("Введите новые минуты (0-60): "))
    adapter.set_time(new_hours, new_minutes)

    print("\n5. ПРОВЕРКА РЕЗУЛЬТАТА")
    print(f"Адаптер показывает: {adapter.display_time()}")
    print(f"Оригинальные аналоговые часы: {adapter.get_analog_clock()}")

    print("\n6. ДЕМОНСТРАЦИЯ РАБОТЫ С ЦИФРОВЫМИ ЧАСАМИ")
    digital = DigitalClock()
    digital.set_time(new_hours, new_minutes)
    print(f"Цифровые часы: {digital}")

    print("\nАдаптер позволяет использовать аналоговые часы")
    print("там же, где ожидаются цифровые часы.")


if __name__ == "__main__":
    main()
```

```
============================================================
ПРОЕКТ ЧАСЫ - АДАПТЕР
============================================================

1. СОЗДАНИЕ АНАЛОГОВЫХ ЧАСОВ
Аналоговые часы: 00:00 (часы: 0.0°, минуты: 0.0°)

2. УСТАНОВКА ВРЕМЕНИ НА АНАЛОГОВЫХ ЧАСАХ
Введите часы (0-12): 11
Введите минуты (0-60): 59
Аналоговые часы: 11:59 (часы: 330.0°, минуты: 354.0°)

3. СОЗДАНИЕ АДАПТЕРА
Адаптер показывает время: 11:59

4. УСТАНОВКА ВРЕМЕНИ ЧЕРЕЗ АДАПТЕР
Введите новые часы (0-23): 18
Введите новые минуты (0-60): 21

5. ПРОВЕРКА РЕЗУЛЬТАТА
Адаптер показывает: 06:21
Оригинальные аналоговые часы: Аналоговые часы: 06:21 (часы: 180.0°, минуты: 126.0°)

6. ДЕМОНСТРАЦИЯ РАБОТЫ С ЦИФРОВЫМИ ЧАСАМИ
Цифровые часы: Цифровые часы: 18:21

Адаптер позволяет использовать аналоговые часы
там же, где ожидаются цифровые часы.
```

Задание 3 (поведенческий паттерн).

Шифрование текстового файла. Реализовать класс-шифровщик текстового файла с поддержкой различных алгоритмов шифрования. Возможные варианты шифрования: удаление всех гласных букв из текста, изменение букв текста на буквы, получаемые фиксированным сдвигом из алфавита (например, шифром буквы а будет являться буква д для сдвига 4 и т.д.), применение операции исключающее или с заданным ключом.

Код программы:

```
"""
Шифрование текстового файла.
Поведенческий паттерн: Strategy (Стратегия).
"""

from abc import ABC, abstractmethod


class EncryptionStrategy(ABC):
    """Абстрактный класс стратегии шифрования."""

    @abstractmethod
    def encrypt(self, text: str) -> str:
        """Зашифровать текст."""

    @abstractmethod
    def decrypt(self, text: str) -> str:
        """Расшифровать текст."""

    @abstractmethod
    def get_name(self) -> str:
        """Получить название алгоритма."""


class VowelRemovalStrategy(EncryptionStrategy):
    """Стратегия: удаление всех гласных букв."""

    def __init__(self):
        self.vowels = "аеёиоуыэюяАЕЁИОУЫЭЮЯaeiouAEIOU"

    def encrypt(self, text: str) -> str:
```

```
        """Удалить гласные."""
        result = ""
        for char in text:
            if char not in self.vowels:
                result += char
        return result

    def decrypt(self, text: str) -> str:
        """Расшифровать невозможно (потеря данных)."""
        return f"[Невозможно восстановить] {text}"

    def get_name(self) -> str:
        return "Удаление гласных"


class CaesarCipherStrategy(EncryptionStrategy):
    """Стратегия: шифр Цезаря (сдвиг)."""

    def __init__(self, shift: int = 4):
        self.shift = shift % 26
        self.alphabet_lower = "abcdefghijklmnopqrstuvwxyz"
        self.alphabet_upper = "ABCDEFGHIJKLMNOPQRSTUVWXYZ"
        self.russian_lower = "абвгдежзийклмнопрстуфхцчшщъыьэюя"
        self.russian_upper = "АБВГДЕЖЗИЙКЛМНОПРСТУФХЦЧШЩЪЫЬЭЮЯ"

    def _shift_char(self, char: str, alphabet: str, shift: int) -> str:
        """Сдвинуть символ в алфавите."""
        if char in alphabet:
            index = alphabet.index(char)
            new_index = (index + shift) % len(alphabet)
            return alphabet[new_index]
        return char

    def encrypt(self, text: str) -> str:
        """Зашифровать сдвигом."""
        result = ""
        for char in text:
            if char in self.russian_lower:
                result += self._shift_char(char, self.russian_lower, self.shift)
            elif char in self.russian_upper:
                result += self._shift_char(char, self.russian_upper, self.shift)
            elif char in self.alphabet_lower:
                result += self._shift_char(char, self.alphabet_lower, self.shift)
            elif char in self.alphabet_upper:
                result += self._shift_char(char, self.alphabet_upper, self.shift)
            else:
                result += char
        return result

    def decrypt(self, text: str) -> str:
        """Расшифровать сдвигом."""
        result = ""
        for char in text:
            if char in self.russian_lower:
                result += self._shift_char(char, self.russian_lower, -self.shift)
            elif char in self.russian_upper:
                result += self._shift_char(char, self.russian_upper, -self.shift)
            elif char in self.alphabet_lower:
                result += self._shift_char(char, self.alphabet_lower, -self.shift)
            elif char in self.alphabet_upper:
                result += self._shift_char(char, self.alphabet_upper, -self.shift)
            else:
                result += char
        return result

    def get_name(self) -> str:
        return f"Цезарь (сдвиг {self.shift})"
```

```
class XorStrategy(EncryptionStrategy):
    """Стратегия: XOR с ключом."""

    def __init__(self, key: str = "secret"):
        self.key = key

    def encrypt(self, text: str) -> str:
        """Зашифровать XOR."""
        result = []
        key_length = len(self.key)
        for i, char in enumerate(text):
            key_char = self.key[i % key_length]
            xor_val = ord(char) ^ ord(key_char)
            result.append(chr(xor_val))
        return "".join(result)

    def decrypt(self, text: str) -> str:
        """Расшифровать XOR (то же самое)."""
        return self.encrypt(text)

    def get_name(self) -> str:
        return f"XOR (ключ: {self.key})"


class FileEncryptor:
    """Класс-шифровщик текстового файла."""

    def __init__(self, strategy: EncryptionStrategy):
        self.strategy = strategy

    def set_strategy(self, strategy: EncryptionStrategy):
        """Сменить стратегию шифрования."""
        self.strategy = strategy

    def encrypt_file(self, input_path: str, output_path: str) -> bool:
        """Зашифровать файл."""
        try:
            with open(input_path, "r", encoding="utf-8") as file_in:
                text = file_in.read()

            encrypted = self.strategy.encrypt(text)

            with open(output_path, "w", encoding="utf-8") as file_out:
                file_out.write(encrypted)

            print(f"Файл зашифрован. Сохранен в: {output_path}")
            return True
        except FileNotFoundError:
            print(f"Ошибка: файл {input_path} не найден")
            return False
        except (IOError, OSError) as error:
            print(f"Ошибка ввода-вывода: {error}")
            return False

    def decrypt_file(self, input_path: str, output_path: str) -> bool:
        """Расшифровать файл."""
        try:
            with open(input_path, "r", encoding="utf-8") as file_in:
                text = file_in.read()

            decrypted = self.strategy.decrypt(text)

            with open(output_path, "w", encoding="utf-8") as file_out:
                file_out.write(decrypted)

            print(f"Файл расшифрован. Сохранен в: {output_path}")
            return True
```

```
        except FileNotFoundError:
            print(f"Ошибка: файл {input_path} не найден")
            return False
        except (IOError, OSError) as error:
            print(f"Ошибка ввода-вывода: {error}")
            return False

    def get_strategy_name(self) -> str:
        """Получить название текущей стратегии."""
        return self.strategy.get_name()


def create_sample_file():
    """Создать пример файла для демонстрации."""
    content = """Привет, мир! Это тестовый файл для шифрования.
Hello, world! This is a test file for encryption.
Цифры: 12345, спецсимволы: !@#$%^&*()"""
    with open("sample.txt", "w", encoding="utf-8") as file_out:
        file_out.write(content)
    print("Создан файл sample.txt")


def main():
    """Основная функция."""
    print("=" * 60)
    print("ШИФРОВАНИЕ ТЕКСТОВОГО ФАЙЛА")
    print("=" * 60)

    create_sample_file()

    print("\nДоступные алгоритмы шифрования:")
    print("1. Удаление гласных")
    print("2. Шифр Цезаря (сдвиг 4)")
    print("3. Шифр Цезаря (свой сдвиг)")
    print("4. XOR с ключом")
    print("5. XOR (свой ключ)")

    choice = input("\nВыберите алгоритм (1-5): ")

    if choice == "1":
        strategy = VowelRemovalStrategy()
    elif choice == "2":
        strategy = CaesarCipherStrategy(4)
    elif choice == "3":
        shift = int(input("Введите сдвиг (1-25): "))
        strategy = CaesarCipherStrategy(shift)
    elif choice == "4":
        strategy = XorStrategy("secret")
    elif choice == "5":
        key = input("Введите ключ: ")
        strategy = XorStrategy(key)
    else:
        print("Неверный выбор")
        return

    encryptor = FileEncryptor(strategy)
    print(f"\nВыбран алгоритм: {encryptor.get_strategy_name()}")

    print("\n1. Зашифровать sample.txt")
    print("2. Расшифровать")
    action = input("Выберите действие (1-2): ")

    if action == "1":
        encryptor.encrypt_file("sample.txt", "encrypted.txt")
        print("\nРезультат шифрования:")
        with open("encrypted.txt", "r", encoding="utf-8") as file_in:
            print(file_in.read()[:200])
    elif action == "2":
```

```
        encryptor.decrypt_file("encrypted.txt", "decrypted.txt")
        print("\nРезультат расшифровки:")
        with open("decrypted.txt", "r", encoding="utf-8") as file_in:
            print(file_in.read()[:200])
    else:
        print("Неверный выбор")


if __name__ == "__main__":
    main()
```

```
============================================================
ШИФРОВАНИЕ ТЕКСТОВОГО ФАЙЛА
============================================================
Создан файл sample.txt

Доступные алгоритмы шифрования:
1. Удаление гласных
2. Шифр Цезаря (сдвиг 4)
3. Шифр Цезаря (свой сдвиг)
4. XOR с ключом
5. XOR (свой ключ)

Выберите алгоритм (1-5): 1

Выбран алгоритм: Удаление гласных

1. Зашифровать sample.txt
2. Расшифровать
Выберите действие (1-2): 1
Файл зашифрован. Сохранен в: encrypted.txt

Результат шифрования:
Првт, мр! т тствй фйл дл шфрвн.
Hll, wrld! Ths s  tst fl fr ncryptn.
Цфр: 12345, спцсмвл: !@#$%^&*()
```

```
============================================================
ШИФРОВАНИЕ ТЕКСТОВОГО ФАЙЛА
============================================================
Создан файл sample.txt

Доступные алгоритмы шифрования:
1. Удаление гласных
2. Шифр Цезаря (сдвиг 4)
3. Шифр Цезаря (свой сдвиг)
4. XOR с ключом
5. XOR (свой ключ)

Выберите алгоритм (1-5): 2

Выбран алгоритм: Цезарь (сдвиг 4)

1. Зашифровать sample.txt
2. Расшифровать
Выберите действие (1-2): 1
Файл зашифрован. Сохранен в: encrypted.txt

Результат шифрования:
Уфмжйц, рмф! Бцт цйхцтжян шднп ипг ьмшфтждсмг.
Lipps, asvph! Xlmw mw e xiwx jmpi jsv irgvctxmsr.
Ьмшфя: 12345, хуйъхмржтпя: !@#$%^&*()
```

Вывод: приобрела навыки применения паттернов проектирования при решении практических задач с использованием языка Python.